

1 TITLE SLIDE

0:00
20 SEC

Good afternoon. I'm Jenul, this is Tam. We're Lab 07, Group 04.

We had to design an MPI simulation of a **wireless sensor network** of EV charging nodes.

I'll cover the topology and the architecture. Tam will cover the analysis.

Breathe. Click to slide 2.

2 TASK 1 – TOPOLOGY TYPE

0:20
60 SEC

We compared five of the topologies from Week 5.

A **complete** topology connects every node to every other one. Node degree grows as $N - 1$, so it scales badly and sends a lot of messages nobody needs.

Linear and **ring** go the other way. Node degree is only one or two, so a node can't reach all four of its neighbours without several hops. That pushes the **diameter** too high for our alert cycle.

Star drops the diameter to two, but every message goes through one hub. For node-to-node traffic that's a bottleneck.

So we chose a two-dimensional **mesh** for the charging nodes. Node degree is two, three or four, which is exactly the four immediate neighbours the spec asks for, and each one is a single hop away.

We kept **star** for the base station, since one server talking to every node is what star is for.

Behind? Drop the star paragraph. The mesh reasoning is what gets marked.

3 TASK 2 – STATIC STRUCTURE (CLASS DIAGRAM)

1:20
40 SEC

This is our static structure.

We map it three ways. **One MPI process per charging node. One MPI process for the base station. And every charging port is a POSIX thread inside its own node's process.**

So we get both kinds of parallelism at once. The port threads share the node's address space and update availability under one mutex, which is **shared memory parallelism**. Anything crossing a process boundary goes as an MPI send or receive instead of a shared variable, and that's the **message passing**.

Slow down on the bold sentence. It is the line that earns the marks.

4 TASK 2 – MESSAGE PASSING (COMMUNICATION DIAGRAM)

2:00
45 SEC

This is one alert cycle. Node four has gone over its port threshold.

The port threads update availability under the mutex first. No MPI yet.

Node four then sends `REQUEST_STATUS` to its four Cartesian neighbours using `MPI_Isend`. We get the neighbours from `MPI_Cart_shift`, which hands back `MPI_PROC_NULL` at the edges, so the corner nodes need no special case.

They answer with `STATUS_RESPONSE`. We use non-blocking receives and `MPI_Waitall` here, because blocking sends would **deadlock** once every node polls on the same tick.

Node four only sends `CAPACITY_ALERT` if all four neighbours are over threshold too. The base station works out the nearest node with free ports and sends back `REDIRECT`.

2:45
35 SEC

On deployment. A rank's port threads share its address space, so we can't split them across machines. Five ports means six threads per rank, and a sixteen-core host takes two ranks. Nine nodes plus the base station puts us at **five machines**.

For bandwidth, one node generates about five kilobits a second. Even at a thousand nodes that's five megabits, which is half a percent of a one gigabit link. So **bandwidth never becomes the bottleneck. The base station does**, and Tam will show you that now.

Hand over. Step back from the screen.

If they ask you

Q&A carries 40% of the mark. Answer in a sentence or two and stop, since the rubric docks you for long answers. Reach for the parallel computing term over the general one.

Why mesh and not a torus or a hypercube?

A torus wraps the edges round, so two stations on opposite sides of the city would count as neighbours. That doesn't mean anything physically. A hypercube gives you log N degree and diameter, but it needs N to be a power of two. The mesh matches how the stations actually sit on a street grid.

Why non-blocking sends?

Every node polls on the same tick. If we used blocking `MPI_Send`, the internal buffer fills up and then every rank is sitting in its send with nobody reaching a receive. That deadlocks. `Isend` and `Irecv` with `MPI_Waitall` let all four exchanges overlap instead.

Why are ports threads and not processes?

The ports in one station share physical state and get read together every cycle. As threads, updating a port is a mutex-protected write. As processes we'd be sending MPI messages for state that never leaves the node.

Could you use OpenMP instead of pthreads?

Yes. The ports are independent, so an `omp parallel for` over the availability array would work. We went with pthreads because each port runs a long-lived simulation loop rather than a bounded parallel region.

How many machines do you actually need?

Five if you want no contention at all. One if you're happy to oversubscribe, since the port threads sit blocked most of the time anyway. Five is the upper bound, not a requirement.

Would this port to a real wireless sensor network?

The protocol would port over fine. It's already point-to-point with no shared state between nodes. What wouldn't is assuming delivery is reliable and in order. A real radio link needs timeouts and retries, and MPI hands us that for free on a cluster.

What is `MPI_PROC_NULL`?

It's the sentinel `MPI_Cart_shift` returns when a neighbour doesn't exist at the edge of the grid. Sends and receives to it return straight away as no-ops, so we don't write special cases for edge nodes.